

Learning From Data

Personal Notes

Mihir Rao

Contents

1	The Learning Problem	2
1.1	From observations to a rule	2
1.2	A simple model: the perceptron	2
1.3	The perceptron learning algorithm	3
1.4	Three kinds of feedback	3
1.5	The necessity of assumptions	4

1 The Learning Problem

Machine learning is useful when a task has an exploitable pattern, but that pattern is difficult to specify as a closed-form rule and examples of the desired behavior are available. Predicting a viewer's movie rating illustrates the idea: observed ratings can reveal latent viewer preferences and movie attributes even when no one can write down the complete relationship in advance.

1.1 From observations to a rule

Let $\mathcal{X}$ be the input space and $\mathcal{Y}$ the output space. A credit applicant, for example, may be represented by an input $x \in \mathcal{X}$ containing age, salary, employment history, and debt, while $y \in \mathcal{Y}$ records whether the applicant is a good or bad credit risk.

Definition 1.1 (Target function). The *target function* $f: \mathcal{X} \rightarrow \mathcal{Y}$ is the unknown rule whose behavior the learner is trying to approximate.

The learner observes a data set

$$\mathcal{D} = \{(x_1, y_1), \dots, (x_N, y_N)\}, \quad y_n = f(x_n),$$

and returns a *final hypothesis* $g: \mathcal{X} \rightarrow \mathcal{Y}$. The goal is not merely to reproduce the recorded outputs, but to make g behave like f on inputs that were not in $\mathcal{D}$.

Two design choices define a learning model:

- The *hypothesis set* $\mathcal{H} = \{h\}$ contains the candidate rules from which the final hypothesis will be selected.
- The *learning algorithm* $\mathcal{A}$ uses the data to choose some $g \in \mathcal{H}$.

Thus the data do not determine a solution by themselves. The hypothesis set expresses which patterns the learner is prepared to consider, and the algorithm determines how evidence selects among them.

1.2 A simple model: the perceptron

Consider binary classification with labels $y \in \{-1, +1\}$ and an input $x = (x_1, \dots, x_d)$. A linear decision rule assigns a weight w_i to each attribute and compares the weighted score with a threshold. Introduce a constant coordinate $x_0 = 1$ and absorb the negative threshold into w_0 .

With the augmented vectors

$$x = (1, x_1, \dots, x_d) \quad \text{and} \quad w = (w_0, w_1, \dots, w_d),$$

the hypothesis becomes

$$h_w(x) = \text{sign}(w^\top x).$$

The equation $w^\top x = 0$ defines a *decision boundary*. Points on one side receive label +1 and points on the other side receive label -1. A labeled data set is *linearly separable* when some w classifies every training point correctly.

Example: In the credit example, a positive weight can represent an attribute that supports approval, while a negative weight can represent one that counts against it. The bias w_0 sets the baseline needed to cross the approval threshold.

1.3 The perceptron learning algorithm

The *perceptron learning algorithm* (PLA) searches for a suitable weight vector. Starting from an initial w , it repeatedly selects a misclassified training example (x_n, y_n) satisfying

$$\text{sign}(w^\top x_n) \neq y_n$$

and performs the update

$$w \leftarrow w + y_n x_n.$$

This update has a direct geometric meaning. Its effect on the selected point's signed score is

$$y_n(w + y_n x_n)^\top x_n = y_n w^\top x_n + \|x_n\|^2.$$

The positive second term moves that example toward the correct side of the decision boundary. PLA alternates between finding an error and making this correction; when no training point is misclassified, the current w defines the final hypothesis.

Note: PLA specifies both parts of the model: linear separators form the hypothesis set, and the iterative correction rule is the learning algorithm. Whether the result also performs well beyond the training data is a separate question.

1.4 Three kinds of feedback

The broad premise of learning is to use observations to uncover an underlying process. Different forms of feedback lead to different learning settings.

Definition: In *supervised learning*, every training input is paired with a desired output. Coin recognition is an example: measurements such as mass and size come with the coin denomination, allowing the learner to infer decision boundaries.

Definition: In *unsupervised learning*, the learner receives inputs without target outputs. It must discover structure such as clusters, common factors, or other regularities in the input distribution.

Definition: In *reinforcement learning*, the learner chooses an output or action and receives a grade for that choice rather than the correct action itself. The feedback evaluates behavior but does not directly reveal what should have been done.

These settings differ in the information supplied to the learner, not in the basic objective of extracting useful structure from observations.

1.5 The necessity of assumptions

A small collection of labeled examples may be compatible with many distinct target functions. A visual pattern puzzle can suggest a natural label for a new shape, but another rule can agree with all displayed labels and predict the opposite answer. Generalization therefore depends on assumptions encoded by $\mathcal{H}$ and $\mathcal{A}$ about which continuations of the observed pattern are plausible.

Source: Yaser S. Abu-Mostafa, Learning From Data, Lecture 1, "The Learning Problem," California Institute of Technology, official course page <https://www.work.caltech.edu/telecourse.html> and slides <https://www.work.caltech.edu/slides/slides01.pdf>.